

Assignment 6: RAG Retrieval-Augmented Generation (RAG)-Machine Learning Knowledge Assistant

1. Introduction

This assignment involved building a Retrieval-Augmented Generation (RAG) system that answers machine learning questions by reading a textbook. Instead of relying on an AI model's training knowledge alone, the system first retrieves relevant passages from a document and then generates an answer grounded strictly in those passages. The source document was Introduction to Machine Learning with Python by Andreas Muller and Sarah Guido, a 392-page PDF. The full pipeline was built in a single Jupyter notebook covering document preprocessing, embedding, vector storage, retrieval, answer generation, and an interactive Q&A interface.

Part 1: Data Understanding and Preprocessing

The PDF was loaded using PyPDF and text was extracted page by page. The book has 392 pages with an average of around 264 words per page. Five pages were blank or image-only and produced no extractable text. Before splitting the text, a five-step cleaning pipeline was applied to fix hyphenated line-breaks, resolve ligature characters such as fi and fl that were encoded as single glyphs, collapse multiple consecutive spaces, and remove embedded control characters like form-feeds.

The cleaned text was split into overlapping chunks using a sentence-aware sliding window, meaning sentences were kept intact as the smallest unit so no chunk would ever cut a sentence in half. Three chunk sizes were tested: 500 tokens with 75 overlap, 750 with 100, and 1000 with 150. The medium configuration of 750 tokens and 100 overlap was chosen, producing 248 chunks. This size gives enough context per chunk for complex ML explanations while staying within the embedding model's token limit. The 100-token overlap ensures context at chunk boundaries is preserved between adjacent chunks.

Part 2: Embedding and Vector Database

Each chunk was converted into a 3072-dimensional vector using Google Gemini's gemini-embedding-001 model. An important design detail is that Gemini supports separate task types for documents and queries: RETRIEVAL_DOCUMENT was used when embedding the stored chunks, and RETRIEVAL_QUERY when embedding the user's question at search time. Using asymmetric task types is recommended by Google for RAG workloads and improves retrieval quality. All vectors were L2-normalised so that cosine similarity equals a simple dot product, making FAISS search faster.

The embeddings were stored in a FAISS IndexFlatIP index, which performs exact brute-force inner-product search. With only 248 chunks the entire index fits comfortably in memory and exact search is preferred over approximate methods since there is no recall trade-off. A practical challenge during

this part was hitting the Gemini free tier rate limit of 100 embedding requests per minute. This was handled by adding a 35-second wait between batches of 25 chunks, completing the full embedding in around six minutes.

Part 3: Retrieval Pipeline

When a user asks a question it is embedded using the RETRIEVAL_QUERY task type and searched against the FAISS index. Three values of k were tested. With k equal to 3, 5, and 7 the top-1 cosine score was identical at 0.6298 for the gradient descent query, confirming the most relevant chunk is always retrieved regardless of k . The mean score dropped slightly from 0.6134 at $k=3$ to 0.6019 at $k=7$, meaning the additional lower-ranked chunks are still reasonably relevant rather than noise. $k=5$ was chosen as the default because it provides around 3,750 tokens of context per query without introducing too many marginal chunks. The best retrieval result was for the bagging and boosting query with a top-1 score of 0.7162, reflecting strong term overlap in the retrieved chunks.

Part 4: Answer Generation

Prompt Design

Retrieved chunks are passed to Gemini inside a structured prompt with four parts: a system role definition, a context block with chunks labelled as Chunk 1 through Chunk k , six strict grounding rules, and the user question. The six rules instruct the model to answer only from the provided context, cite which chunks were used with inline notation, and explicitly say when the context does not contain enough information. The generation temperature was set to 0.2 to keep answers close to the source material, and top_p was set to 0.85.

Sample Query Results

Five test questions were run through the full pipeline. The overfitting and regularization query produced a detailed well-grounded answer with multiple chunk citations and a vocabulary overlap of 79.4% with the retrieved context, which was the strongest result because the book covers regularization in depth. The bagging and boosting query returned a partial answer, correctly noting that the term bagging is not used explicitly in the retrieved chunks even though random forests and gradient boosting are covered.

The remaining three queries on gradient descent, convolutional neural networks, and the vanishing gradient problem all returned the fallback message stating the context was insufficient. This is the correct behaviour. The book is introductory and does not cover these deep learning topics in enough depth for the retrieved chunks to support a full answer. The hallucination guard in the prompt worked as intended by preventing the model from generating an answer beyond what the context supports.

Part 5: End-to-End Application

All components were integrated into a single `rag_pipeline` function and an interactive Q&A loop. The user can type any question and the system retrieves relevant chunks, builds a grounded prompt, calls Gemini, and prints the answer with the chunk references. The session runs in a loop until the user types quit or exit. Rate limiting was handled by parsing the exact retry wait time from the 429 error response and waiting that long before retrying automatically, so the session recovers without crashing.

7. Conclusion

The RAG system works well for questions directly covered in the book and correctly handles out-of-scope questions by falling back to a clear message rather than hallucinating. The main limitation is the introductory nature of the source book, which does not cover deep learning topics in sufficient depth for the retrieval to return useful context. Building each component manually rather than using a framework like LangChain made it possible to fully understand each stage and debug issues like rate limiting and prompt grounding effectively.